2025 International Conference on Machine Learning and Applications (ICMLA) | 979-8-3315-5980-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICMLA66185.2025.00225

Generation of Programmatic Rules for Document Forgery Detection Using Large Language Models

1st Valentin Schmidberger
Stuttgart Media University
Stuttgart, Germany
schmidberger@hdm-stuttgart.de

1st Manuel Eberhardinger
Stuttgart Media University
Stuttgart, Germany
eberhardinger@hdm-stuttgart.de

2nd Setareh Maghsudi
Ruhr-University Bochum
Bochum, Germany
setareh.maghsudi@rub.de

3rd Johannes Maucher
Stuttgart Media University
Stuttgart, Germany
maucher@hdm-stuttgart.de

***Abstract*—Document forgery poses a growing threat to legal, economic, and governmental processes, requiring increasingly sophisticated verification mechanisms. One approach involves the use of plausibility checks, rule-based procedures that assess the correctness and internal consistency of data, to detect anomalies or signs of manipulation. Although these verification procedures are essential for ensuring data integrity, existing plausibility checks are manually implemented by software engineers, which is time-consuming. Recent advances in code generation with large language models (LLMs) offer new potential for automating and scaling the generation of these checks. However, adapting LLMs to the specific requirements of an unknown domain remains a significant challenge. This work investigates the extent to which LLMs, adapted on domain-specific code and data through different fine-tuning strategies, can generate rule-based plausibility checks for forgery detection on constrained hardware resources. We fine-tune open-source LLMs, Llama 3.1 8B and OpenCoder 8B, on structured datasets derived from real-world application scenarios and evaluate the generated plausibility checks on previously unseen forgery patterns. The results demonstrate that the models are capable of generating executable and effective verification procedures. This also highlights the potential of LLMs as scalable tools to support human decision-making in security-sensitive contexts where comprehensibility is required.**

***Index Terms*—Deep Learning, Large Language Models, Domain-Specific Fine-Tuning, Document Forgery**

I. Introduction

The rapid advancement of digitization and globalization has led to a notable increase in document forgery, which poses major challenges for both the private and public sectors. The proliferation of digital tools and interconnected global markets facilitates the creation and dissemination of counterfeit documents, with reported cases rising by 15.9% between 2019 and 2023 [9]. This emphasizes the urgent need for innovative solutions.

Automated AI-based systems are increasingly crucial for detecting patterns and anomalies in document verification processes [10], [21] and also for automating procedures to comply with legal regulations [7]. The main objective of this work is the adaptation of pre-trained large language models (LLMs) to generate plausibility checks, represented as Python code, that support the detection of document forgeries. Plausibility checks are rule-based procedures that assess the correctness and internal consistency of data, e.g., whether the issuing date of the document is an official working day of the issuing country; another check verifies if the document number is unique in the entire database.

This involves leveraging different fine-tuning strategies to incorporate domain-specific knowledge and developing mechanisms to generalize across emerging forgery patterns. Furthermore, privacy concerns require the use of models that operate entirely on locally stored data, since closed-source models, such as GPT-4 [15] from OpenAI, process user input externally, which conflicts with data protection guidelines in security sensitive contexts. Another prerequisite for the use of AI models in law enforcement is the need for comprehensibility of the prediction, which we circumvent by using LLMs to generate program code that is inherently interpretable and can therefore be checked for correctness by experts before these checks are used to support the decision-making processes of employees.

Figure 1 shows an overview of the methodology. We start with the existing document verification system, which consists of a Python code base, plausibility checks, and documents stored in the database. From this we create four datasets for fine-tuning. The fine-tuning is divided into two phases: The first phase focuses on fine-tuning the models on the code base dataset and instruction dataset, while the second phase involves further fine-tuning on the existing plausibility checks and document properties of the database. We use four LLMs from two model families, Llama 3.1 8B (base/instruct) [8] and OpenCoder 8B (base/instruct) [12] and fine-tune them with Low-Rank Adaptation [11] on constrained hardware resources, namely a single Nvidia RTX 6000 48GB GPU. To augment existing data, Llama 3.3 70B is used to generate synthetic data following the method proposed in [8], [23]. In the first phase, the models are evaluated on their ability to generate executable code for the document verification system, while in the second phase the generated plausibility checks are evaluated on their performance in detecting unseen document forgeries. Our code and data are part of proprietary software and therefore cannot be made available as open source.

This paper is structured as follows: We start with related work in Sect. II, followed by the background of the document verification system that defines the context of this work (Sect. III). Afterwards, the methodology for creating the datasets for phase 1 (Sect. IV) and phase 2 (Sect. V) is explained. Each phase includes an evaluation and discussion of the results,

1946-0759/25/$31.00 ©2025 IEEE
DOI 10.1109/ICMLA66185.2025.00225

Authorized licensed use limited to: REVA UNIVERSITY. Downloaded on October 09,2026 at 06:59:52 UTC from IEEE Xplore. Restrictions apply.

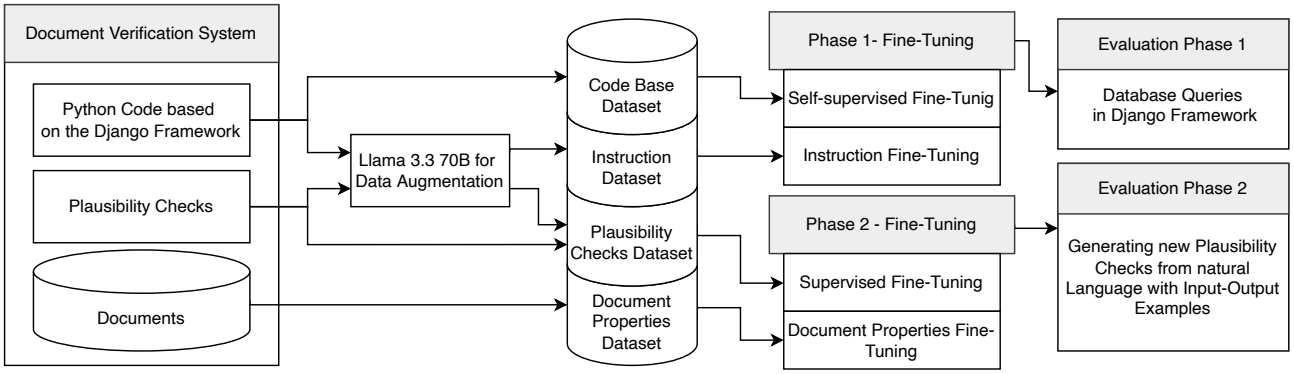


Fig. 1. Overview of the methodology. We start with the existing document verification system, which consists of a Python code base, plausibility checks, and documents stored in the database. From this we create four datasets for fine-tuning and use Llama 3.3 70B for data augmentation. The fine-tuning is divided into two phases: The first phase focuses on fine-tuning the models on the code base dataset and instruction dataset, while the second phase involves further fine-tuning on the existing plausibility checks and document properties of the database. In the first phase, the models are evaluated on their ability to generate executable code for the document verification system, whereas in the second phase the generated plausibility checks are evaluated on their performance in detecting unseen document forgeries.

followed by an overall discussion of this study in Sect. VI. We conclude the paper in Sect. VII.

II. Related Work

The efficiency of supervised fine-tuning of LLMs and data augmentation is highlighted in multiple papers [20], [14], [25], [8], [23]. Wang et al. [23] showed that generating instructions with the same LLM that is also fine-tuned, significantly improves its performance; similar to the creation of the instruction dataset in our work. Zhou et al. showed in *LIMA: Less Is More for Alignment* [25] that the quality of training data often outweighs quantity, especially for specialized applications. For code generation of IoT software, Shen et al. [20] have shown that small, fine-tuned models trained on domain-specific data perform better than generic LLMs in typical IoT coding tasks. Li et al. [14] further found that although LLMs have difficulty synthesizing code from input-output examples, it is possible to solve complex programming by example problems after supervised fine-tuning on these kinds of tasks.

Recently, large language models in text forensics have been adapted for author profiling, i.e., with the objective of identifying personal characteristics of the author based on writing styles [4]. In [19], the Llama 3.1 8B model was fine-tuned for Q&A on digital forensic research papers and artifacts and conducted a user survey with forensic professionals. Scanlon et al. [18] evaluated the potential impact of ChatGPT for forensic tasks and concluded that many applications are unsuitable as the evidence is uploaded to the service provider, and also warned against hallucinations of LLMs. A comprehensive survey [24] analyzed where LLMs are used in digital forensics and also outlined their risks. In this survey, no other LLM was used as a code generation tool to support the decision-making process for law enforcement.

An area in digital forensics, where LLMs have been more widely adopted is cyber security [1], e.g., cyber threat intelligence [16], vulnerability detection [22] or penetration testing [5]. Recently, Cherif et al. [3] introduced a new benchmark for evaluating LLMs in cyber security tasks within the context of incident response, which include realistic forensic challenges as well as disk and memory forensics.

To the best of our knowledge, we are the first to use LLMs in the forensic analysis of identification and related documents to generate plausibility checks, rule-based procedures represented in Python, to assess the internal consistency and correctness of data, supporting decision-making in security-sensitive contexts where comprehensibility is essential.

III. Document Verification System

The context of this work is a document verification system that is based on the Django framework[1] and implemented in Python. The system is an AI-assisted processing platform for all types of documents, such as passports, birth certificates or driver's licenses from all over the world, and is used to check documents for forgeries. Scans of printed documents are uploaded to the platform and various AI models are used to help users detect forgeries faster and more reliably. The entire code base is proprietary software and, therefore, not available online, which is a major challenge for pre-trained and instruction-tuned LLMs when creating executable code in this domain; this is also visible in the following experiments as without fine-tuning there is no available knowledge about this project online and, consequently, not in the training data of open-source models. The complete project consists of 109,824 lines of code and 78 custom database models based on the Django database-abstraction API. Generating executable Django queries for the database is a significant challenge for LLMs, as the code base and model names of the document verification system are in English, but the values stored in the database for filtering and checking properties are partly in German and partly in English. The existing

[1]https://www.djangoproject.com/

Authorized licensed use limited to: REVA UNIVERSITY. Downloaded on October 09,2026 at 06:59:52 UTC from IEEE Xplore. Restrictions apply.

plausibility checks are implemented manually in Python and evaluate the correctness and internal consistency of the data. Some checks are quite simple, only checking that the issuing date of the document is not in the future, while others are more complicated. The most complicated check retrieves all documents of the same type from the database and trains a linear regression model to check if the document number correlates with the issuing date. An imaginary check for data integrity is shown in Listing 1, which checks if driver licenses from Germany are of plastic – the check for "Kunststoff" in line 16 – if they are issued in between March 2000 and March 2010.

```
1  def f(document):
2      import datetime as dt
3      from document_project.models import (DocumentCategory,
           DocumentElement,DocumentElementEvaluation)
4      from plausibility_checks.plausibility_checks import
           RuleDoesNotApply
5      if (document.issuing_country.code != 'DE'
6          or document.doc_type.doc_category !=
               DocumentCategory.objects.get(doc_category='Fü
               hrerschein, national')
7          or document.issuing_date is None
8          or document.issuing_date < dt.date(year=2000, month
               =3, day=1)
9          or document.issuing_date > dt.date( year=2010,
               month=3, day=31)):
10         raise RuleDoesNotApply("Gegebenes Dokument ist
               nicht für die Regel relevant.")
11     material_evals = DocumentElementEvaluation.objects.
           filter(document=document,
12             check_result__document_element=DocumentElement.
                   objects.get(name='Material'))
13     result_dict = {}
14     for i, material_eval in enumerate(material_evals):
15         result_dict[f'Material {i}'] = material_eval.
               check_result.check_result_category
16     result = material_eval.exclude(
           check_result__check_result_category__iexact='
           Kunststoff').count() > 0
17     return result, result_dict
```

Listing 1. Example of a plausibility check, which checks if the material of a given document is of plastic. This check is only executed for German driver's licenses and if the issuing date of the document is between March 2000 and March 2010.

There are still a few important constraints we would like to clarify. The document verification system is designed to assess whether documents are authentic or potentially forged. However, the plausibility checks described here represent only a small part of the overall system. They serve merely as indicators of possible inconsistencies or signs of falsification. A warning triggered by a plausibility check does not necessarily mean that a document is forged; it may still be genuine. In the end, these checks are intended to support experts by providing them with indications of areas that they should review before drawing a conclusion about the document in question.

Furthermore, our system is specifically designed for scanned images of printed documents. Electronic documents are not taken into account, nor are electronic fingerprints of documents or PDF signatures.

IV. Phase One: Python Code Fine-Tuning

A. Fine-Tuning Process

In the first phase, two different fine-tuning strategies derived from the Python code base of the document verification system are evaluated for Llama 3.1 8B [8] and OpenCoder 8B [12]. The primary objective is to ensure that the models develop a fundamental understanding of the code structure, allowing them to comprehend the relation of the Python classes and data flows within the system. The key research question is whether training on raw code base provides a solid foundation for generating more precise plausibility checks in the subsequent phase.

1) Objective and Methodology: The main objective of this phase is to embed both semantic and structural understanding of the document verification system code base into the language models. This enables the models to accurately interpret the syntax and semantics of the specific code ecosystem, translate user queries into executable code by correctly leveraging existing functions and libraries, generate relevant code snippets to address technical challenges or analyze particular properties of documents.

The fine-tuning process consists of several key steps. First, in the data preparation stage, relevant code is extracted, cleaned, and tokenized to construct a diverse and representative training dataset. Next, during model training, the LLMs are fine-tuned using this dataset, with ongoing experimental evaluation and validation to monitor learning progress. Finally, the evaluation phase focuses on verifying the syntactic and semantic correctness of the code generated by the models.

A comprehensive analysis of the impact of this pre-training will be conducted after the completion of the two fine-tuning phases in order to assess its influence on the effectiveness of the model in the subsequent phase for generating plausibility checks.

2) Dataset Preparation: The training dataset was derived from the document verification system code base, methodically segmented into smaller, logically coherent sections. This segmentation facilitated targeted training and efficient processing by the model. Modules, classes, and functions were identified as central structural elements and separated accordingly. The preprocessing steps included the removal of repetitive comments, code formatting, and tokenization. The resulting dataset consisted of 109,824 lines of code, which were segmented into 310 chunks using a maximum sequence length of 8192 tokens.

3) Instruction Dataset Generation: To enhance the efficiency of fine-tuning smaller models, the more powerful Llama 3.3 70B model was employed to generate synthetic data based on the existing code base. This approach follows the methodology outlined in [8], where synthetic data generated by a larger model improves the learning quality of smaller models.

The backend code base is used as the basis to generate prompts in the so-called *Instruction-Format* [23] for fine-tuning using the bigger Llama 3.3 70B model. We were able to

Authorized licensed use limited to: REVA UNIVERSITY. Downloaded on October 09,2026 at 06:59:52 UTC from IEEE Xplore. Restrictions apply.

deploy the 70B model on a single Nvidia RTX 6000 48GPU by using nested 4-bit quantization [6]. Each dataset entry follows a uniform structure of task, input, and expected output:

- Instruction: Description of the task
- Input: Input data or parameters (optional)
- Output: Expected code

We let the Llama 3.3 model create instructions for the code blocks from the Python code base. If no specific input data is required, the input remains `None`. The instruction dataset was segmented into 21,368 chunks, each with a maximum length of 8192 tokens. This dataset is therefore 700 times the size of the code base dataset.

B. Experiments

1) Evaluation Data: To evaluate the fine-tuned models, various coding queries were defined and prototypes of sample solutions were created using Llama 3.3 70B. All prototypes were reviewed and corrected by experts before being used in the evaluation. These queries were categorized into three levels based on their complexity: Low, Medium and High. The complexity was determined by the depth of the queries and the number of nested structures within the code base. The following Table I shows examples of queries and Python code snippets categorized by complexity level.

2) Experimental Setup: We compare several fine-tuning strategies against baseline models, which were not adapted to the specific task. These baseline models serve as a reference point for assessing performance improvements. Three levels of fine-tuning are considered: (1) self-supervised fine-tuning, on the Python code base from the project; (2) instruction fine-tuning, based on generated instruction dataset derived from the code base; and (3) a combined fine-tuning approach that applies both fine-tuning strategies sequentially. The evaluation is performed using ten queries per complexity level, and each query is executed five times to mitigate the effect of randomness. A generation is successful if it exactly matches the reference implementation.

3) Evaluation Metrics: We employ four different evaluation metrics. The *Success Rate* measures the percentage of responses generated that match the reference solution. Only the output of the programs is used for comparison and only exact matches count as correct solutions. This is similar to calculating the accuracy in classification tasks. The other two metrics are based on the Gestalt-pattern matching algorithm [17], which calculates the similarity between two string sequences S_1 and S_2:

$$Sim(S_1, S_2) = \frac{2M}{|S_1| + |S_2|} \tag{1}$$

where M is the number of consecutive matching characters and $|S_1|$ and $|S_2|$ are the length of the string sequences. This metrics complements the Success Rate since it also accounts for partial matches in sequences. We define the *Code Match* to assess the syntactic similarity of the generated program with the reference solution. We decided to check for syntactic similarity, since the custom code base has many specific class names and object values, so this metric indicates if the LLMs have acquired knowledge about the code base of the document verification system. The Code Match calculates the average similarity of all programs with the corresponding reference solution. The *Output Match* evaluates the similarity between the generated output and the reference output. Similarly to the Code Match, the Output Match is the average of the Gestalt-pattern similarity of all generated outputs for the test cases. We also report the *pass@k* metric introduced in [2], which measures if at least one of k generated completions correctly solves a given coding task.

4) Results: Table II presents the final evaluation of phase 1. There are notable performance differences visible between the models and their respective fine-tuning strategies. OpenCoder consistently achieves high success rates across all complexity levels, whereas Llama shows less consistency, particularly on high-complexity tasks. The OpenCoder Instruct model, fine-tuned with both the instruction dataset and plain code, achieves a 100% success rate in low- and mid-level queries and 70% in high-level queries. In contrast, the Llama Instruct model, fine-tuned solely on the instruction dataset, reaches success rates of 54% for low-level, 64% for mid-level, and only 20% for high-level queries. These results suggest that the combined fine-tuning approach used with OpenCoder enhances the model's ability to generalize across varying levels of complexity and better interpret both structural and semantic aspects of the code.

The Pass@5 metric indicates whether at least one of five generated completions correctly solves a given coding task (1 if true, else 0). Table III compares the best OpenCoder (Instruction Dataset and Code Base Fine-Tuning) and Llama (Instruction Dataset Fine-Tuning) models, which also shows that OpenCoder solves more test cases than Llama.

The superior performance of OpenCoder can likely be attributed to its pre-training on code-specific datasets, enabling it to capture recurring syntactic patterns and programming patterns more effectively. In contrast, the Llama model, being a general-purpose language model trained on broad textual data, lacks specialized expertise in programming only task. Overall, the OpenCoder Instruct model with combined fine-tuning demonstrates the most reliable and best performance across all evaluation scenarios.

V. Phase Two: Plausibility Checks

A. Fine-Tuning Process

In the second phase, the LLMs, previously fine-tuned on the document verification system code, are further trained on existing plausibility checks and document properties. The objective is to optimize the models to generate new, technically sound, and relevant plausibility checks. Since both instruct models outperformed their base counterparts, we focus our experiments in this phase on the instruct models.

1) Objective and Methodology: The focus is on developing validation procedures that identify forgeries based on specific document properties. The plausibility checks are defined by functions operating on document objects. The methodology

Authorized licensed use limited to: REVA UNIVERSITY. Downloaded on October 09,2026 at 06:59:52 UTC from IEEE Xplore. Restrictions apply.

TABLE I
EXAMPLE QUERIES AND GENERATED PYTHON CODE SNIPPETS CATEGORIZED BY COMPLEXITY LEVEL FOR ACCESSING DIFFERENT DOCUMENT PROPERTIES.

Complexity	Query	Code

Low	Generate Python code for the project, that retrieves and prints all barcodes.	`from example.barcode_detection.models import Barcode`
		`barcodes = Barcode.objects.all()`
		`print(barcodes)`

Mid	Generate Python code for the project, that retrieves and updates all DocumentElementField objects and prints them out.	`from example.models import DocumentElementField`
		`fields = DocumentElementField.objects.filter(field_type='example.DateField')`
		`for field in fields:`
		`  field.field_type = 'example.DateField'`
		`  field.save()`
		`  print(field)`

High	Get all visa requirements for countries that have a specific visa requirement information (e.g., identifier=1) and print them out.	`from example.models import Country, VisaRequirementInformation, VisaRequirement`
		`visa_req_info = VisaRequirementInformation.objects.get(identifier=1)`
		`countries = Country.objects.filter(visarequirementinformation=visa_req_info)`
		`visa_requirements = VisaRequirement.objects.filter(country_of_entry__in=`
		`  countries)`
		`print(visa_requirements)`

TABLE II
PERFORMANCE OF THE BASE MODELS AND AFTER VARIOUS FINE-TUNING ON THE CODE, THE INSTRUCTION DATASET OR BOTH. L STANDS FOR LLAMA 3.1 8B, OC FOR OPENCODER 8B, B FOR THE BASE MODEL AND I FOR THE INSTRUCTION TUNED MODEL. WE REPORT THE SUCCESS RATE *SR*, OUTPUT MATCH *OM* AND CODE MATCH *CM*

		Baseline			Code			Instruction Dataset			Code & Instruction		
Model	**Level**	**SR**	**OM**	**CM**	**SR**	**OM**	**CM**	**SR**	**OM**	**CM**	**SR**	**OM**	**CM**
L-B	Low	2	16	12	0	30	6	8	10	53	20	21	46
	Mid	2	42	12	2	44	7	18	22	50	18	20	43
	High	0	5	12	0	4	12	4	10	38	8	15	36
L-I	Low	4	5	40	6	8	48	54	55	60	44	44	66
	Mid	6	21	33	24	24	44	64	62	52	42	48	49
	High	8	15	41	8	14	32	20	28	33	18	23	39
OC-B	Low	4	5	37	30	30	47	80	82	76	80	90	70
	Mid	6	10	37	20	50	41	30	32	38	60	62	61
	High	8	15	34	0	6	44	50	43	56	50	51	55
OC-I	Low	0	3	75	40	40	70	100	100	85	100	100	89
	Mid	0	5	57	40	45	62	80	80	69	100	100	65
	High	10	16	55	20	23	57	30	31	65	70	68	50

TABLE III
PASS@5 FOR CHECK EXECUTION – EACH ENTRY INDICATES WHETHER ≥ 1 OUT OF 5 SOLUTIONS PASSED THE TEST CASE (T1-T10).

Model	Level	T1	T2	T3	T4	T5	T6	T7	T8	T9	T10
Llama	Low	1	0	1	1	1	0	1	1	1	1
	Mid	1	1	1	1	1	0	1	1	0	1
	High	0	1	0	1	1	0	0	1	1	0
OpenC.	Low	1	1	1	1	1	1	1	1	1	1
	Mid	1	1	1	1	1	1	1	1	1	1
	High	1	1	0	1	1	1	1	0	1	0

involves fine-tuning the LLM on existing plausibility checks, enabling the generation of functional validation mechanisms from natural language descriptions, such as "For German driving licenses issued between March 2000 and March 2010, the material must be plastic." for Listing 1. This description is paired with input and output examples, i.e. the document as input and the output of the executed check, which is always a Boolean value indicating whether the check was triggered, with a Python dictionary for additional information. The goal is to ensure the model can independently create code for validation mechanisms, considering both structural and content aspects of documents. The final evaluation compares model-generated checks with manually created reference solutions to assess the model's ability to develop precise and technically correct validation rules.

2) Dataset Preparation: The dataset preparation involves collecting and processing documents for training and evaluation of plausibility checks, including both genuine and forged examples. A key component of the fine-tuning process is 35 expert-created plausibility checks, each extended with 10 input-output examples, totaling 350 examples. These examples ensure the model understands the logic of plausibility checks and can apply them to specific documents.

Authorized licensed use limited to: REVA UNIVERSITY. Downloaded on October 09,2026 at 06:59:52 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
EXAMPLE VALUES FOR PROPERTIES OF A SPECIFIC DOCUMENT.

Document ID	Property	Original Value	Translation
8713426	Document Type	Reisepass	Passport
	Issuing Country	Deutschland	Germany
	Assessment	Fälschung	Forged
	Document Number	X12345	
	Issuing Date	2020-05-01	

To improve the generalizability and diversity of the dataset, synthetic plausibility checks were generated, similar to the instruction dataset in phase 1. The Llama 3.3 70B model was used to create new, similar checks based on existing plausibility checks. This method produced 100 additional synthetic plausibility checks, each enriched with specific input-output examples, increasing the total to 450 examples. The dataset comprises 100 generated entries in addition to the 350 existing examples, totaling in 450 chunks (with a chunk length of 8192 tokens).

3) Document Properties as Dataset: A dataset with document-specific properties was created to further extend the model's knowledge. For example, a specific type of identification document may only be issued between 2008 and 2012. This fine-tuning enables the model to process and interpret such context-specific information accurately. An example of such a dataset is shown in Table IV.

These properties help improve the accuracy of model predictions by focusing on specific document characteristics. The document properties dataset comprises 100,000 entries, resulting in 100,000 chunks (with a chunk length of 8192 tokens).

B. Experiments

1) Evaluation Data: To prepare the plausibility checks used to assess the performance of the models, specific properties of the provided data were artificially altered. These adjustments are solely for evaluation purposes, testing the models' ability to detect manipulations. Characteristics such as material, security technology, printing technique, or personalization were synthetically modified. These artificial adjustments form the basis for creating plausibility checks formulated in natural language. An example of such a synthetic change is shown in Table V. For evaluation, 10 plausibility checks per difficulty level (Mid, High) were defined. A check was considered successful if the generated program's output matched the reference output exactly.

2) Experimental Setup: In phase 2 we use as a baseline the OpenCoder Instruct and Llama Instruct models. These are also the basis for further fine-tuning on the plausibility checks. To assess the influence of the previous phase, we fine-tune the best OpenCoder and Llama model from phase 1 on the plausibility checks, and also on document properties. The same evaluation metrics as in phase 1 are used (see Sect. IV-B3). Similarly to the previous experiments, each test is repeated five times to mitigate the effects of randomness.

TABLE V
SYNTHETIC CHANGES IN DOCUMENT MATERIALS. FOR CLARITY WE USE THE ENGLISH TRANSLATION OF THE VALUES.

Material	Type	Evaluation
Cover	Plastic	deviating
Page 1	Paper	matches specification
Page 3	Paper	matches specification
Page 4	Paper	matches specification
Page 5	Paper	matches specification
Page 6/7	Paper	matches specification

a) Pre-experimental Observations:: A major issue identified during the analysis of the generated plausibility checks is that LLMs often incorrectly generate the following code section:

```
from plausibility_checks . plausibility_checks import
    RuleDoesNotApply
if (document.example != 'example'):
    raise RuleDoesNotApply("Given document is not relevant
        for this check.")
```

Listing 2. The code section at the beginning of plausibility checks that checks if a given document is relevant for this check.

This checks whether a given document is relevant for the plausibility check. See Listing 1 for a more detailed example, which only executes the check if a German driver's license was issued between March 2000 and March 2010, and otherwise throws an exception that the given document is not relevant. This code section only occurs in a few plausibility checks, making it difficult for LLMs to learn the correct implementation, even though the rest of the check is implemented correctly. As a result, the error rate of plausibility checks increases significantly, necessitating a more robust evaluation approach. To address this issue, we define two evaluation methods:

- **Exact**: This is a strict evaluation where no modification of the generated program code is made.
- **Regex**: This evaluation uses a regex expression to modify the code snippet presented in Listing 2 to print a message instead of raising an exception. With this modification, the output of the check is evaluated without interrupting its execution.

3) Results: Table VI summarizes the performance of the models in generating plausibility checks after the second fine-tuning phase. The table compares their performance across two complexity levels (Mid, High) using two evaluation methods, highlighting the impact of domain-specific fine-tuning on the models' ability to generate functionally correct and executable validation routines. The evaluation results indicate that Llama and OpenCoder exhibit different strengths depending on the fine-tuning approach. The combination of fine-tuning the best code model of phase 1 on document properties and plausibility checks leads to significant performance improvements. Llama outperforms OpenCoder, particularly in integrating document properties and fine-tuning on plausibility checks. This is attributed to its broader, generalized pre-training, which enhances its understanding of document structures, since the

Authorized licensed use limited to: REVA UNIVERSITY. Downloaded on October 09,2026 at 06:59:52 UTC from IEEE Xplore. Restrictions apply.

TABLE VI

PERFORMANCE OF THE GENERATED PLAUSIBILITY CHECKS FOR THE BASE MODELS AND AFTER VARIOUS FINE-TUNING ON THE PLAUSIBILITY CHECK DATASET (DENOTED BY FT), THE DOCUMENT PROPERTIES (DENOTED BY D) OR BOTH. L STANDS FOR LLAMA 3.1 8B, OC FOR OPENCODER 8B, I FOR THE INSTRUCTION TUNED MODEL AND BC FOR THE BEST CODE MODEL FROM PHASE 1. WE REPORT THE SUCCESS RATE *SR*, OUTPUT MATCH *OM* AND CODE MATCH *CM* FOR THE EXACT EVALUATION AND THE ADJUSTED REGEX EVALUATION.

		Exact			Regex		
Model	**Level**	**SR**	**OM**	**CM**	**SR**	**OM**	**CM**
L-I	Low	0	9	3	0	10	2
	Mid	0	8	9	0	7	7
L-I FT	Low	0	35	2	4	32	5
	Mid	2	27	1	4	28	4
L-BC+FT	Low	14	51	14	26	49	30
	Mid	22	35	27	34	35	35
L-BC+D+FT	Low	26	67	25	44	60	44
	Mid	20	42	19	32	43	32
OC-I	Low	0	10	2	0	10	0
	Mid	0	9	10	0	7	0
OC-I FT	Low	0	11	0	0	11	0
	Mid	0	14	0	0	15	0
OC-BC+FT	Low	10	52	10	20	52	22
	Mid	20	45	19	50	43	46
OC-BC+D+FT	Low	10	44	13	20	50	19
	Mid	30	37	34	20	32	22

document properties are in German. This is reasonable since Llama 3.1 is a multilingual model for seven languages, including German. OpenCoder also improved its performance through targeted fine-tuning on plausibility checks, but it still lagged behind Llama. Notably, OpenCoder shows stronger performance in the mid-complexity checks.

The impact of different fine-tuning and multistage fine-tuning is particularly visible. Baseline models, without any fine-tuning, demonstrate low success rates for all evaluation metrics. Introducing fine-tuning on plausibility checks alone results in moderate gains in the output match; however, this strategy alone does not produce substantial improvements. In contrast, applying targeted fine-tuning on plausibility checks to the best-performing models from Phase 1 –those already fine-tuned on backend code – yields significant performance improvements. The most effective approach combines all stages, starting with the best model from the previous phase and all fine-tuning datasets. This comprehensive strategy achieves the highest overall performance, with Llama showing the greatest improvement at this stage due to its enhanced ability to interpret document properties.

Table VII shows the pass@5 metric for the best Llama (Best Code, Documents, Checks Finetuned) and OpenCoder (Best Code, Checks Finetuned) models. For low complexity, the Llama model solves more than twice as many checks as OpenCoder. For mid complexity, OpenCoder solves one more check.

VI. DISCUSSION, LIMITATIONS & FUTURE WORK

This study explored the adaptation of pre-trained large language models for generating document verification mechanisms based on Python code on constrained hardware resources. The results demonstrate that fine-tuning LLMs on domain-specific datasets significantly enhances their ability to generate executable validation rules.

TABLE VII

PASS@5 FOR PLAUSIBILITY CHECK EXECUTION – EACH ENTRY INDICATES WHETHER $\geq$1 OUT OF 5 SOLUTIONS PASSED THE TEST CASE (C1-C10).

Model	**Level**	**C1**	**C2**	**C3**	**C4**	**C5**	**C6**	**C7**	**C8**	**C9**	**C10**
Llama	Low	1	1	0	1	1	1	1	1	1	1
	Mid	1	1	1	1	1	0	1	0	0	0
OpenC.	Low	1	0	1	1	0	0	0	0	0	1
	Mid	0	1	1	1	1	1	0	1	0	1

The fine-tuning process revealed distinct strengths in each model. OpenCoder performed exceptionally well in tasks involving code-specific processing, while Llama 3.1 demonstrated superior generalization capabilities, especially in generating plausibility checks where document properties in German need to be combined with the database queries. The multistage fine-tuning strategy, initially focused on the project code and later on actual plausibility checks, significantly improved the models' performance compared to non-finetuned baseline models. This multistage approach enhanced the models' ability to generate precise and executable plausibility checks, making them more effective for document verification.

Nevertheless, working with LLM is a challenge, especially due to its black-box nature, which makes it difficult to evaluate the efficiency of certain adjustments or data preparation. Although evaluation metrics can help to measure efficiency, the process is time-consuming. Hardware limitations restricted fine-tuning to smaller 8B models and prevented testing larger models that could provide better results. In addition, handling sensitive data in a complex project environment required caution, but also provided an opportunity to develop innovative solutions. Another problem of LLMs is the need for retraining, when new information about document forgeries is added or the code base of the document verification system is updated. Several promising directions exist for future research. Retrieval-Augmented Generation [13] could allow models to dynamically incorporate external, up-to-date knowledge bases (e.g., known forgery patterns or updates of document security features), potentially enhancing validation accuracy without constant retraining.

Other promising research directions are the integration of LLMs with computer vision methods to enable analysis of visual document features (layout, seals, fonts) for more robust forgery detection. Deploying these models in real-time systems within relevant operational environments (e.g., law enforcement or border control) would provide critical insights into practical performance, scalability, and usability.

VII. CONCLUSION

The findings of this study demonstrate the potential of fine-tuned LLMs for generating programmatic rules for document

Authorized licensed use limited to: REVA UNIVERSITY. Downloaded on October 09,2026 at 06:59:52 UTC from IEEE Xplore. Restrictions apply.

forgery detection. A multistage supervised fine-tuning approach, initially focusing on the document verification system code and subsequently on specific plausibility checks and document properties, proved effective in adapting the Llama 3.1 and OpenCoder models to this domain-specific task. The results show that fine-tuned models can generate executable and relevant validation code, significantly outperforming the baseline models. Llama models showed better generalization, particularly when fine-tuned on document properties, while OpenCoder excelled after code-specific fine-tuning.

This work lays the foundation for automated document verification systems. The success of fine-tuning confirms that models can generate both code fragments and complete validation routines with reasonable accuracy, offering a scalable tool to improve security processes.

ACKNOWLEDGMENT

We extend our gratitude to the Document Forensics Department of the Baden Württemberg State Office of Criminal Investigations for their invaluable assistance in providing the data of the documents essential to this project. Special recognition is owed to Rolf Fauser, whose personal dedication and insightful expertise were instrumental in facilitating our work.

REFERENCES

[1] Atlam, H.F.: Llms in cyber security: Bridging practice and education. Big Data and Cognitive Computing **9**(7), 184 (2025)

[2] Chen, M., Tworek, J., Jun, H., ..., Zaremba, W.: Evaluating large language models trained on code (7/7/2021), http://arxiv.org/pdf/2107.03374v2, corrected typos, added references, added authors, added acknowledgements

[3] Cherif, B., Bisztray, T., Dubniczky, R.A., Aldahmani, A., Alshehhi, S., Tihanyi, N.: Dfir-metric: A benchmark dataset for evaluating large language models in digital forensics and incident response. arXiv preprint arXiv:2505.19973 (2025)

[4] Cho, S.H., Kim, D., Kwon, H.C., Kim, M.: Exploring the potential of large language models for author profiling tasks in digital text forensics. Forensic Science International: Digital Investigation **50**, 301814 (2024)

[5] Deng, G., Liu, Y., Mayoral-Vilches, V., Liu, P., Li, Y., Xu, Y., Zhang, T., Liu, Y., Pinzger, M., Rass, S.: Pentestgpt: An llm-empowered automatic penetration testing tool. arXiv preprint arXiv:2308.06782 (2023)

[6] Dettmers, T., Pagnoni, A., Holtzman, A., Zettlemoyer, L.: Qlora: Efficient finetuning of quantized llms. Advances in neural information processing systems **36**, 10088–10115 (2023)

[7] Eberhardinger, M., Takenaka, P., Grießhaber, D., Maucher, J.: Anonymization of documents for law enforcement with machine learning. In: 2025 IEEE Symposium Series on Computational Intelligence (SSCI). pp. 1–7. IEEE (2025)

[8] Grattafiori, A., Dubey, A., Jauhri, A., et.al.: The llama 3 herd of models (7/31/2024), http://arxiv.org/pdf/2407.21783v3, s. 1-34

[9] Groß, Heidi, BKA-KI12-1: Polizeiliche kriminalstatistik 2023 ausgewählte zahlen im Überblick [police crime statistics 2023 an overview of selected statistics] https://www.bka.de/SharedDocs/Downloads/DE/Publikationen/PolizeilicheKriminalstatistik/2023/FachlicheBroschueren/IMK-Bericht.pdf?__blob=publicationFile&v=6

[10] Gupta, S., Kumar, M.: Forensic document examination system using boosting and bagging methodologies. Soft Computing **24**(7), 5409–5426 (2020)

[11] Hu, E.J., yelong shen, Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W.: LoRA: Low-rank adaptation of large language models. In: International Conference on Learning Representations (2022), https://openreview.net/forum?id=nZeVKeeFYf9

[12] Huang, S., Cheng, T., Liu, J.K., Hao, J., Song, L., Xu, Y., Yang, J., Liu, J.H., Zhang, C., Chai, L., Yuan, R., Zhang, Z., Fu, J., Liu, Q., Zhang, G., Wang, Z., Qi, Y., Xu, Y., Chu, W.: Opencoder: The open cookbook for top-tier code large language models (11/7/2024), http://arxiv.org/pdf/2411.04905v2, s. 1-14

[13] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.t., Rocktäschel, T., et al.: Retrieval-augmented generation for knowledge-intensive nlp tasks. Advances in neural information processing systems **33**, 9459–9474 (2020)

[14] Li, W.D., Ellis, K.: Is programming by example solved by llms? (6/12/2024), http://arxiv.org/pdf/2406.08316v2

[15] OpenAI: Gpt-4 technical report (3/15/2023), http://arxiv.org/pdf/2303.08774v6

[16] Perrina, F., Marchiori, F., Conti, M., Verde, N.V.: Agir: Automating cyber threat intelligence reporting with natural language generation. In: 2023 IEEE International Conference on Big Data (BigData). pp. 3053–3062. IEEE (2023)

[17] Ratcliff, J.W., Metzener, D.E., et al.: Pattern matching: The gestalt approach. Dr. Dobb's Journal **13**(7), 46 (1988)

[18] Scanlon, M., Breitinger, F., Hargreaves, C., Hilgert, J.N., Sheppard, J.: Chatgpt for digital forensic investigation: The good, the bad, and the unknown. Forensic Science International: Digital Investigation **46**, 301609 (2023)

[19] Sharma, B., Ghawaly, J., McCleary, K., Webb, A.M., Baggili, I.: Forensicllm: A local large language model for digital forensics. Forensic Science International: Digital Investigation **52**, 301872 (2025)

[20] Shen, L., Yang, Q., Huang, X., Ma, Z., Zheng, Y.: Gpiot: Tailoring small language models for iot program synthesis and development (3/2/2025), http://arxiv.org/pdf/2503.00686v1

[21] Takenaka, P., Eberhardinger, M., Grießhaber, D., Maucher, J.: Classification of inkjet printers based on droplet statistics. In: 2024 International Joint Conference on Neural Networks (IJCNN). pp. 1–7. IEEE (2024)

[22] Tamberg, K., Bahsi, H.: Harnessing large language models for software vulnerability detection: A comprehensive benchmarking study. IEEE Access (2025)

[23] Wang, Y., Kordi, Y., Mishra, S., Liu, A., Smith, N.A., Khashabi, D., Hajishirzi, H.: Self-instruct: Aligning language models with self-generated instructions. In: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). pp. 13484–13508 (2023)

[24] Yin, Z., Wang, Z., Xu, W., Zhuang, J., Mozumder, P., Smith, A., Zhang, W.: Digital forensics in the age of large language models. arXiv preprint arXiv:2504.02963 (2025)

[25] Zhou, C., Liu, P., Xu, P., Iyer, S., Sun, J., Mao, Y., Ma, X., Efrat, A., Yu, P., Yu, L., et al.: Lima: Less is more for alignment. Advances in Neural Information Processing Systems **36**, 55006–55021 (2023)

Authorized licensed use limited to: REVA UNIVERSITY. Downloaded on October 09,2026 at 06:59:52 UTC from IEEE Xplore. Restrictions apply.